

# **SPORTS TOURNAMENT MANAGEMENT SYSTEM**

## **A Project Report**

**CBSE Class XII (INFORMATION PRACTICE)**

**By**

**YOUR\_NAME**

**Board\_Seat\_No.**

**Under the esteemed guidance of**

**IP\_SUBJECT TEACHER NAME**



**CENTRAL RAILWAY SCHOOL AND JR. COLLEGE KALYAN**

***(Affiliated to CBSE New Delhi Affiliation No. 1180013)***

**KALYAN, 421301**

**MAHARASHTRA**

**2021-2022**



## CERTIFICATE

This is to certify that the Project entitled \_\_\_\_\_  
is a Bonafide work done by \_\_\_\_\_ of  
class X IT, session 2025-2026 in partial fulfilment of CBSE's AISSCE  
Examination 2025 and has been carried out under my direct supervision  
and guidance. He/She has taken proper care and shown utmost sincerity  
in completion of this project. I certify this project is up to my expectation  
and as per the guidelines issued by the CBSE.

\_\_\_\_\_  
Internal Examiner

\_\_\_\_\_  
External Examiner

\_\_\_\_\_  
Principal

\_\_\_\_\_  
School Stamp

# CONTENTS

## **1. Introduction**

1.1 Aim

1.2 Objective

## **2. Requirement Analysis**

2.1 Technologies Used

2.3 Software Requirement

## **3. Design and Coding**

3.1 Source Code

3.2 CSV

3.3 Output

## **4. Conclusion**

## **5. Bibliography**

# 1. Introduction

Sports play an important role in promoting teamwork, leadership, and discipline. Managing tournaments manually often leads to confusion in scheduling, recording scores, and ranking teams. To overcome these issues, the Sports Tournament Management System helps in managing data related to teams, players, matches, and scores efficiently. It provides analytical results, such as rankings and performance summaries, in tabular and graphical formats. Using this system, tournament organizers can save time, reduce human errors, and make data-driven decisions with ease.

## 1.1 Aim

The main aim of this project is to design and develop a Python-based Sports Tournament Management System that stores, manages, and analyzes tournament data. The system should help users view fixtures, input scores, and generate rankings automatically. It also aims to visualize performance using bar charts for better understanding.

## 1.2 Objectives

- To create a computerized system for managing sports tournament data.
- To maintain records of teams, players, matches, and scores systematically.
- To analyze and display team rankings and player performance using Python libraries.
- To visualize match results using bar charts and tables.
- To ensure easy data retrieval and modification using a database or CSV files.
- To demonstrate integration of **Python**, **Pandas**, **Matplotlib**, and **MySQL** for data analysis.

## 2. Requirement Analysis

### 2.1 Technology Used

- **Programming Language:** Python
- **Libraries:** Pandas, Matplotlib, MySQL Connector
- **Database:** MySQL / CSV
- **Data Handling:** CSV and SQL queries
- **Visualization:** Bar charts and tables

### 2.2 Software Requirements

- Python 3.9 or above
- MySQL Server and Workbench (for database)
- IDLE (for coding) / Jupyter Notebook
- Matplotlib and Pandas installed
- CSV data files

### 3. Design

The project consists of four main entities:

1. Teams – stores team details such as **ID**, name, and city.
2. Players – stores player details and their respective team **IDs**.
3. Matches – records scores and results of each match.
4. Tournaments – stores tournament details like name and year.

Data is analyzed using Pandas DataFrames, and visualized using Matplotlib charts.

#### 3.1 Source Code :-

```
# ----- Import Libraries -----  
  
import pandas as pd  
import matplotlib.pyplot as plt  
from sqlalchemy import create_engine, text  
import warnings  
warnings.filterwarnings('ignore')  
  
# ----- Database Connection (SQLAlchemy - No Warning!) -----  
  
# Format: mysql+mysqlconnector://username:password@host:port/database  
engine = create_engine('mysql+mysqlconnector://root:1234@localhost:3306/sportsdb')
```

# ----- Functions -----

def show\_teams():

query = "SELECT \* FROM teams"

df = pd.read\_sql(query, engine)

print("\n 🏆 Teams List:")

print(df.to\_string(index=False))

def show\_players():

query = """SELECT p.player\_id, p.player\_name, t.team\_name

FROM players p JOIN teams t ON p.team\_id = t.team\_id"""

df = pd.read\_sql(query, engine)

print("\n 🧑 Players List:")

print(df.to\_string(index=False))

def show\_matches():

query = """SELECT m.match\_id, t1.team\_name AS Team1, t2.team\_name AS Team2,

m.team1\_score, m.team2\_score, t3.team\_name AS Winner

FROM matches m

JOIN teams t1 ON m.team1\_id = t1.team\_id

JOIN teams t2 ON m.team2\_id = t2.team\_id

JOIN teams t3 ON m.winner\_id = t3.team\_id"""

df = pd.read\_sql(query, engine)

print("\n 🏆 Match Results:")

print(df.to\_string(index=False))

def team\_rankings():

query = """SELECT winner\_id AS team\_id, COUNT(winner\_id) AS Wins

FROM matches GROUP BY winner\_id ORDER BY Wins DESC"""

```

df = pd.read_sql(query, engine)
teams = pd.read_sql("SELECT * FROM teams", engine)
df = df.merge(teams, on="team_id")
df = df[['team_name', 'Wins']]

print("\n 🏆 Team Rankings:")
print(df.to_string(index=False))

# 🎨 BEAUTIFUL GRAPH WITH GRADIENT COLORS 🎨
plt.style.use('seaborn-v0_8-darkgrid')
fig, ax = plt.subplots(figsize=(12, 7))

# Gradient colors from dark blue to light cyan
colors = plt.cm.viridis(range(len(df)))

# Create bars with gradient
bars = ax.bar(df['team_name'], df['Wins'], color=colors,
              edgecolor='white', linewidth=2, alpha=0.9)

# Add value labels on top of bars
for bar in bars:
    height = bar.get_height()
    ax.text(bar.get_x() + bar.get_width()/2., height,
            f'{int(height)}',
            ha='center', va='bottom', fontsize=14, fontweight='bold', color='darkblue')

# Styling
ax.set_title(" 🏆 Team Rankings - Number of Wins 🏆 ",

```

```

        fontsize=20, fontweight='bold', pad=20, color='darkblue')
ax.set_xlabel("Teams", fontsize=14, fontweight='bold', color='navy')
ax.set_ylabel("Number of Wins", fontsize=14, fontweight='bold', color='navy')
ax.tick_params(axis='x', rotation=45, labels=12)
ax.tick_params(axis='y', labels=12)

# Grid styling
ax.grid(True, alpha=0.3, linestyle='--', linewidth=0.8)
ax.set_axisbelow(True)

# Background color
ax.set_facecolor('#f0f8ff')
fig.patch.set_facecolor('white')

plt.tight_layout()
plt.show()

def player_statistics():
    query = "SELECT * FROM players"
    df = pd.read_sql(query, engine)
    print("\n 🧑 Player Statistics:")
    print(df.to_string(index=False))

# 🎨 BONUS: Beautiful Player Stats Graph! 🎨
if len(df) > 0 and 'runs' in df.columns and 'wickets' in df.columns:
    show_graph = input("\n 📊 Would you like to see a graph? (y/n): ").lower()
    if show_graph == 'y':
        fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

```

```

# Runs Graph

colors1 = plt.cm.Oranges(range(len(df)))

bars1 = ax1.barh(df['player_name'], df['runs'], color=colors1,
                 edgecolor='darkred', linewidth=2)

ax1.set_title("🏏 Player Runs", fontsize=16, fontweight='bold', color='darkred')
ax1.set_xlabel("Runs", fontsize=12, fontweight='bold')
ax1.grid(True, alpha=0.3, axis='x')
ax1.set_facecolor('#fff5ee')


# Wickets Graph

colors2 = plt.cm.Greens(range(len(df)))

bars2 = ax2.barh(df['player_name'], df['wickets'], color=colors2,
                 edgecolor='darkgreen', linewidth=2)

ax2.set_title("🎯 Player Wickets", fontsize=16, fontweight='bold',
color='darkgreen')
ax2.set_xlabel("Wickets", fontsize=12, fontweight='bold')
ax2.grid(True, alpha=0.3, axis='x')
ax2.set_facecolor('#f0fff0')


plt.tight_layout()

plt.show()


def add_team():

    print("\n➕ Add New Team")
    print("-" * 40)
    team_name = input("Enter Team Name: ").strip()
    city = input("Enter City: ").strip()

```

```

if team_name and city:

    try:

        from sqlalchemy import text

        with engine.connect() as conn:

            # Get the max team_id and add 1

            result = conn.execute(text("SELECT COALESCE(MAX(team_id), 0) + 1 AS
next_id FROM teams"))

            next_id = result.fetchone()[0]

            conn.execute(text(f"INSERT INTO teams (team_id, team_name, city)
VALUES ({next_id}, '{team_name}', '{city}'))

            conn.commit()

            print(f"✅ Team '{team_name}' added successfully!")

        except Exception as e:

            print(f"❌ Error adding team: {e}")

        else:

            print(f"❌ Team name and city cannot be empty!")

```

```

def add_player():

    print("\n➕ Add New Player")

    print("-" * 40)

    # Show available teams

    teams_df = pd.read_sql("SELECT * FROM teams", engine)

    print("\n📋 Available Teams:")

    print(teams_df.to_string(index=False))

    player_name = input("\nEnter Player Name: ").strip()

    team_id = input("Enter Team ID: ").strip()

```

```

runs = input("Enter Runs (default 0): ").strip() or "0"
wickets = input("Enter Wickets (default 0): ").strip() or "0"

if player_name and team_id:
    try:
        from sqlalchemy import text
        with engine.connect() as conn:
            # Get the max player_id and add 1
            result = conn.execute(text("SELECT COALESCE(MAX(player_id), 100) + 1
AS next_id FROM players"))
            next_id = result.fetchone()[0]

            conn.execute(text(f"INSERT INTO players (player_id, player_name, team_id,
runs, wickets) VALUES ({next_id}, '{player_name}', {team_id}, {runs}, {wickets})"))
            conn.commit()

            print(f"✅ Player '{player_name}' added successfully!")
    except Exception as e:
        print(f"❌ Error adding player: {e}")
    else:
        print(f"❌ Player name and team ID cannot be empty!")

# ----- Main Menu -----
while True:
    print("\n" + "="*50)

    print("🏏 SPORTS TOURNAMENT MANAGEMENT SYSTEM 🏏".center(50))
    print("="*50)

    print("1. 📋 Show Teams")
    print("2. 👤 Show Players")

```

```
print("3. 🏏 Show Matches")
print("4. 🏆 Team Rankings (with Graph)")
print("5. 📊 Player Statistics (with Graph)")
print("6. ➕ Add New Team")
print("7. ➕ Add New Player")
print("8. ✖ Exit")
print("="*50)
```

```
try:
```

```
    choice = int(input("Enter your choice (1-8): "))
```

```
    if choice == 1:
```

```
        show_teams()
```

```
    elif choice == 2:
```

```
        show_players()
```

```
    elif choice == 3:
```

```
        show_matches()
```

```
    elif choice == 4:
```

```
        team_rankings()
```

```
    elif choice == 5:
```

```
        player_statistics()
```

```
    elif choice == 6:
```

```
        add_team()
```

```
    elif choice == 7:
```

```
        add_player()
```

```
    elif choice == 8:
```

```
        print("\n✅ Thank you for using Sports Management System!")
```

```
        print("👋 Exiting Program... Goodbye!")
```

break

else:

print(" ❌ Invalid choice! Please enter a number between 1-8.")

except ValueError:

print(" ❌ Invalid input! Please enter a valid number.")

### 3.2 output :-

```
*IDLE Shell 3.11.2*
File Edit Shell Debug Options Window Help

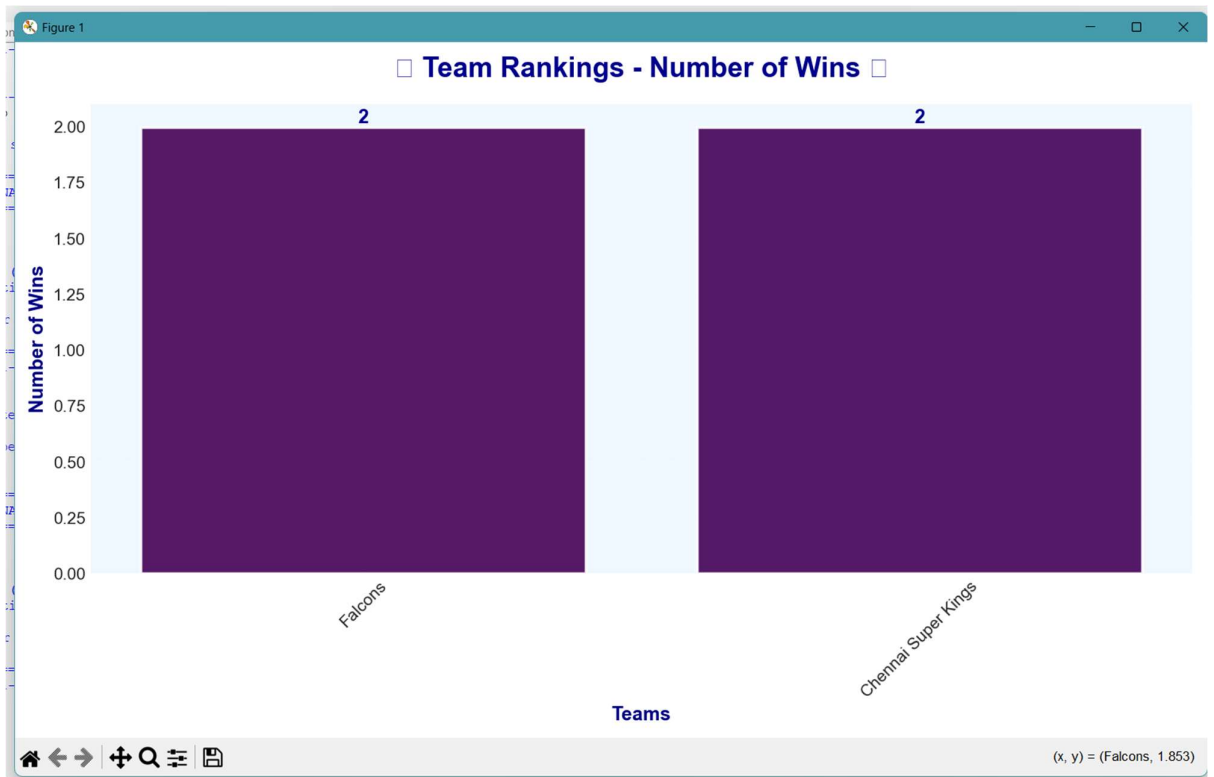
=====
d SPORTS TOURNAMENT MANAGEMENT SYSTEM d
=====
1. [ ] Show Teams
2. [x] Show Players
3. [x] Show Matches
4. [x] Team Rankings (with Graph)
5. [x] Player Statistics (with Graph)
6. [ ] Add New Team
7. [ ] Add New Player
8. [x] Exit
=====
Enter your choice (1-8): 6

+ Add New Team
=====
Enter Team Name: rcb
Enter City: beng
[ ] Team 'rcb' added successfully!

=====
d SPORTS TOURNAMENT MANAGEMENT SYSTEM d
=====
1. [ ] Show Teams
2. [x] Show Players
3. [x] Show Matches
4. [x] Team Rankings (with Graph)
5. [x] Player Statistics (with Graph)
6. [ ] Add New Team
7. [ ] Add New Player
8. [x] Exit
=====
Enter your choice (1-8): 1

d Teams List:
team_id      team_name    city
1            Falcons    Delhi
2 Chennai Super Kings Chennai
3            rcb      beng

=====
d SPORTS TOURNAMENT MANAGEMENT SYSTEM d
=====
1. [ ] Show Teams
2. [x] Show Players
3. [x] Show Matches
```



## **Conclusion**

The Sports Tournament Management System simplifies the process of managing sports events by digitalizing data entry, storage, and analysis. Using Python Pandas for data management and Matplotlib for visualization makes data interpretation easy and accurate. The project successfully integrates Python with MySQL to handle structured data efficiently.

It demonstrates how programming can automate real-world problems like tournament management, enhance productivity, and ensure data accuracy. The system can be extended further to include features such as player-wise statistics, automatic fixture generation, and web-based dashboards. Overall, this project provides a practical understanding of database connectivity, data analysis, and visualization in a real-world application setting.

## **Bibliography**

### **Books:**

- CBSE Informatics Practices (Class XII) - NCERT
- “Python for Everybody” by Charles Severance
- “Learning Python” by Mark Lutz

### **Websites / References:**

- <https://pandas.pydata.org>
- <https://matplotlib.org>
- <https://www.mysql.com>
- <https://www.w3schools.com/python>
- <https://docs.python.org/3>